

A close-up photograph of a yellow and white striped snake, possibly a corn snake, coiled around its head. The snake's body is wrapped in a protective coil, with its head in the center. The background is a soft, out-of-focus yellow and white pattern, matching the snake's skin. The lighting is warm and focused on the snake's head.

# Python Control Structures

## CHAPTER TWO

PIYUSH PANT

# If Statement

---

**Definition:** Executes a block of code if the condition is true.

**Syntax:**

```
if condition:  
    statement(s)
```

**Example:**

```
age = 20  
if age >= 18:  
    print("You are eligible to vote.")
```

**Output:** You are eligible to vote.

# Match Statement

**Definition:** Used to compare a variable against multiple cases (introduced in Python 3.10).

## Syntax:

```
match variable:
    case value1:
        action1
    case value2:
        action2
    Case _:
        defaultaction
```

## Example:

```
day = "Monday"
match day:
    case "Monday":
        print("Start of the week")
    case "Friday":
        print("Weekend coming")
    case _:
        print("Regular day")
```

**Output:** Start of the week

# Break Statement

Exits the nearest enclosing loop immediately. Break out of loop .

## Example:

```
for i in range(5):  
    if i == 3:  
        break  
    print(i)
```

## Output:

0  
1  
2



# Continue Statement

---

Skips the current iteration and continues with the next.

## **Example:**

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

## **Output:**

```
0  
1  
3  
4
```

# Loops

---

- Repeats a block of code multiple times.
- **Types:**
  - while loop
  - for loop

# While Loop

---

**Definition:** Repeats while the condition is true.

**Syntax & Example:**

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

**Output:** 1 2 3 4 5

# For Loop

**Definition:** Iterates over a sequence (like list, range).

**Syntax & Example:**

```
for i in range(1, 6):  
    print(i)
```

**Output:** 1 2 3 4 5

# Nested Loop

**Definition:** A loop inside another loop.

**Example:**

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print(i, j)
```

**Output:**

1 1

1 2

1 3

2 1

2 2

2 3

3 1

3 2

3 3

# Infinite Loop

---

A loop that runs forever unless externally stopped.

## **Example:**

```
while True:
```

```
    print("This is an infinite loop")
```

**Note:** Use with caution; typically includes `break` inside to stop conditionally.

# Use Case (Infinite Loop)

1. You want a program that keeps responding to user input until the user types "bye".

while True:

```
    msg = input("You: ")
```

```
    if msg.lower() == "bye":
```

```
        print("Bot: Goodbye!")
```

```
        break
```

```
    print("Bot: I heard you say", msg)
```

***Used in:*** customer support bots, AI assistants, or classroom help apps.

# Use Case (Infinite Loop)

## 2. Monitoring a Sensor (like Temperature or Motion)

In devices like smart thermostats or motion detectors:

while True:

```
    temperature = read_temperature() #this is function which calculates temperature
```

```
    if temperature > 30:
```

```
        print("Warning: High temperature!")
```

**Used in:** customer support bots, AI assistants, or classroom help apps.

# Use Case (Infinite Loop)



## ATM Interface Simulation



A loop for ATM-like interaction where users can check balance, withdraw, or exit:



*Used in:* banking apps,



infinite loops help keep programs responsive and running continuously until the user—or a condition—decides otherwise.

```
balance = 1000
```

```
while True:
```

```
    print("1. Check Balance\n2. Withdraw\n3. Exit")
```

```
    option = input("Choose an option: ")
```

```
    if option == '1':
```

```
        print("Balance:", balance)
```

```
    elif option == '2':
```

```
        amount = int(input("Enter amount: "))
```

```
        if amount <= balance:
```

```
            balance -= amount
```

```
            print("Withdrawn:", amount)
```

```
        else:
```

```
            print("Insufficient funds")
```

```
    elif option == '3':
```

```
        break
```

# Python Data Structures

Lists : A **list** is a collection of items that is **ordered**, **changeable**, and allows **duplicate** values.

Example:

```
fruits = ['apple', 'banana', 'cherry']  
print(fruits[0]) # Output: apple
```

## List Slicing

```
numbers = [10, 20, 30, 40, 50]  
print(numbers[1:4]) # Output: [20, 30, 40]
```

# in Operator (List)

Used to check if item is in list or not

```
if 'banana' in fruits:
```

```
    print("Yes, banana is in the list.")
```



| Method        | Description                       | Example                   |
|---------------|-----------------------------------|---------------------------|
| append(x)     | Adds item at the end              | fruits.append('orange')   |
| index(x)      | Returns index of first occurrence | fruits.index('banana')    |
| insert(i,x)   | Inserts at given index            | fruits.insert(1, 'grape') |
| sort()        | Sorts list in ascending order     | fruits.sort()             |
| remove(x)     | Removes first occurrence          | fruits.remove('banana')   |
| reverse()     | Reverses the list                 | fruits.reverse()          |
| min() / max() | Returns smallest/largest value    | min([4, 1, 7]) → 1        |

# List Comprehension

A **shorter** way to create new lists from existing ones.

- List comprehension is a **concise and elegant way to create new lists** by applying an expression to each item in an iterable (like a list, tuple, or range).
- It's more compact and often faster than writing a loop to generate a list.

## General Syntax

python

```
[expression for item in iterable if condition]
```

- **expression**: What you want each item to become.
- **item**: The variable representing each element in the iterable.
- **iterable**: A sequence you loop through (like a list or range).
- **if condition (optional)**: Filter items based on this condition.

# Why Use List Comprehension?

- It's **shorter and more readable** than traditional loops.
- Often **faster in performance** for creating lists.
- Makes code **more Pythonic** and elegant.

## Equivalent to a for-loop

- **Traditional Loop:**

```
squares = []  
for x in range(5):  
    squares.append(x**2)
```

- **List Comprehension:**

```
squares = [x**2 for x in range(5)]
```

# List Comprehension

## Example 1: Square of numbers

```
squares = [x**2 for x in range(5)]  
print(squares) # Output: [0, 1, 4, 9, 16]
```

## Example 2: Even numbers (with conditions , only even numbers are added to list )

```
evens = [x for x in range(10) if x % 2 == 0]  
print(evens) # Output: [0, 2, 4, 6, 8]
```

## Example 3 : Nested List (3\*3 table )

```
matrix = [[i * j for j in range(3)] for i in range(3)]
```

Creates a 3x3 multiplication table.

## Two-Dimensional Lists

A **list of lists**, used to represent matrices or tables.

### Example: 2D List

```
matrix = [[1, 2], [3, 4], [5, 6]]  
print(matrix[1][0]) # Output: 3
```

### Accessing All Elements

```
for row in matrix:  
    for val in row:  
        print(val, end=' ')
```

# Tuples

A **tuple** is similar to a list but **immutable** (cannot be changed after creation).

```
point = (3, 4)
```

```
print(point[0]) # Output: 3
```

## Tuples vs Lists

- Lists use `[]` and are mutable.
- Tuples use `()` and are immutable.

# Dictionaries

- A **dictionary** is a built-in Python data type used to store **data in key-value pairs**.
- Each **key** is unique and is used to access its corresponding **value**.
- Dictionaries are **mutable**, i.e., you can change, add, or remove elements after creation

## **syntax**

```
my_dict = {key1: value1, key2: value2, ...}
```

# Dictionaries

## Creating a dictionary

```
person = {  
    "name": "Alice",  
    "age": 30,  
    "city": "Kathmandu"  
}
```

## Accessing values

```
print(person["name"])  # Output: Alice  
print(person.get("age")) # Output: 30
```

## Adding new Value

```
person["email"] = "alice@example.com"
```

## Modifying Existing Values

```
person["age"] = 31
```

## Removing Elements

```
del person["city"]      # Removes city
```

```
email = person.pop("email") # Removes and returns the value of email
```

```
person.clear()          # Empties the dictionary
```

# Common Dictionary Methods

| Method   | Description                                     | Example                                    | Output                                                              |
|----------|-------------------------------------------------|--------------------------------------------|---------------------------------------------------------------------|
| get(key) | Returns value of key; None if key doesn't exist | person.get("name")                         | 'Alice'                                                             |
|          |                                                 | person.get("email")                        | None                                                                |
| keys()   | Returns all keys                                | person.keys()                              | dict_keys(['name', 'age', 'city'])                                  |
| values() | Returns all values                              | person.values()                            | dict_values(['Alice', 30, 'Kathmandu'])                             |
| items()  | Returns key-value pairs as tuples               | person.items()                             | dict_items([('name', 'Alice'), ('age', 30), ('city', 'Kathmandu')]) |
| update() | Updates or adds key-value pairs                 | person.update({"age": 31})                 | Updates age to 31                                                   |
|          |                                                 | person.update({"email": "alice@mail.com"}) | Adds email key                                                      |
| pop(key) | Removes key and returns its value               | person.pop("city")                         | 'Kathmandu'                                                         |

# Sets

- A **set** is an **unordered** collection of **unique elements**.
- Sets are **mutable** (can be changed), but elements must be **immutable** (e.g., strings, numbers, tuples).

can use:

**Numbers:** 10, 5.5

**Strings:** 'hello'

**Tuples:** (1, 2)

But you **cannot** use:

**Lists:** [1, 2]

**Dictionaries:** {'a': 1}

**Other sets:** {1, 2} while creating sets

## Creating a set

```
my_set = {1, 2, 3}
```

```
empty_set = set()    # Use set() for an emptyset, {} creates a dictionary
```

## Adding Elements

```
my_set.add(4)
```

## Removing Elements

```
my_set.remove(2)    # Removes 2; raises error if not present
```

```
my_set.discard(5)    # Removes 5 if present; no error if not
```

```
my_set.clear()       # Empties the set
```

Given two sets:  $A = \{1, 2, 3\}$   $B = \{3, 4, 5\}$

| Operation       | Syntax          | Result                  |
|-----------------|-----------------|-------------------------|
| Union           | $A \cup B$      | $\{1, 2, 3, 4, 5\}$     |
| Intersection    | $A \cap B$      | $\{3\}$                 |
| Difference      | $A - B$         | $\{1, 2\}$              |
| Symmetric Diff. | $A \oplus B$    | $\{1, 2, 4, 5\}$        |
| Subset          | $A \subseteq B$ | True if $A \subseteq B$ |
| Superset        | $A \supseteq B$ | True if $A \supseteq B$ |

# Strings

A **string** in Python is a **sequence of characters** enclosed in **single (' ')**, **double (" ")**, or **triple quotes (''' or """)**.

Strings are **immutable** — once created, they cannot be changed.

## String Slicing

```
s = "HelloWorld"
print(s[0:5])      # Output: Hello
print(s[:5])       # Output: Hello
print(s[5:])       # Output: World
print(s[-5:])      # Output: World
```



| Method    | Description                                  | Example             | Output |
|-----------|----------------------------------------------|---------------------|--------|
| isalpha() | True if all characters are letters           | "Hello".isalpha()   | True   |
|           |                                              | "Hi123".isalpha()   | False  |
| isdigit() | True if all characters are digits            | "12345".isdigit()   | True   |
|           |                                              | "123abc".isdigit()  | False  |
| isalnum() | True if all characters are letters or digits | "abc123".isalnum()  | True   |
|           |                                              | "abc 123".isalnum() | False  |
| islower() | True if all characters are lowercase         | "hello".islower()   | True   |
|           |                                              | "Hello".islower()   | False  |
| isupper() | True if all characters are uppercase         | "WORLD".isupper()   | True   |
|           |                                              | "World".isupper()   | False  |
| isspace() | True if string contains only whitespace      | " ".isspace()       | True   |
|           |                                              | " a ".isspace()     | False  |



# String Searching

```
s = "Python is awesome"
```

```
print(s.find("is"))    # Output: 7 (position of 'is')
```

```
print(s.startswith("Py")) # Output: True
```

```
print(s.endswith("some")) # Output: True
```

```
print("is" in s)       # Output: True
```

# String Manipulation Methods

---

## Method

`upper()`

`lower()`

`capitalize()`

`title()`

`replace(old, new)`

`split(delimiter)`

`join(list)`

`strip()`

## Description

Converts all characters to uppercase

Converts all characters to lowercase

Capitalizes the first character

Capitalizes the first character of each word

Replaces substring with another

Splits string into list by delimiter

Joins list elements into a string

Removes leading and trailing whitespaces

# Changing Case

```
text = "hello world"
```

```
print(text.upper())    # "HELLO WORLD"
```

```
print(text.lower())    # "hello world"
```

```
print(text.capitalize()) # "Hello world"
```

```
print(text.title())    # "Hello World"
```

```
print(text.swapcase())  # "HELLO WORLD" becomes "hello world",  
and vice versa
```

# Replacing and Removing Character

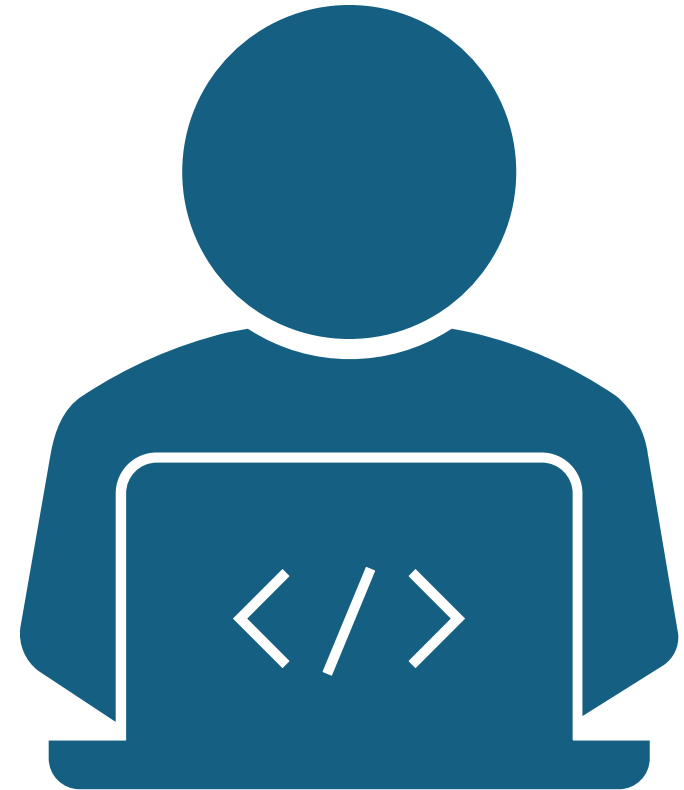
```
text = "I love Python programming"
```

```
print(text.replace("Python", "Java")) # "I love Java programming"
```

```
print(text.replace(" ", "_"))        # "I_love_Python_programming"
```

## **To remove characters:**

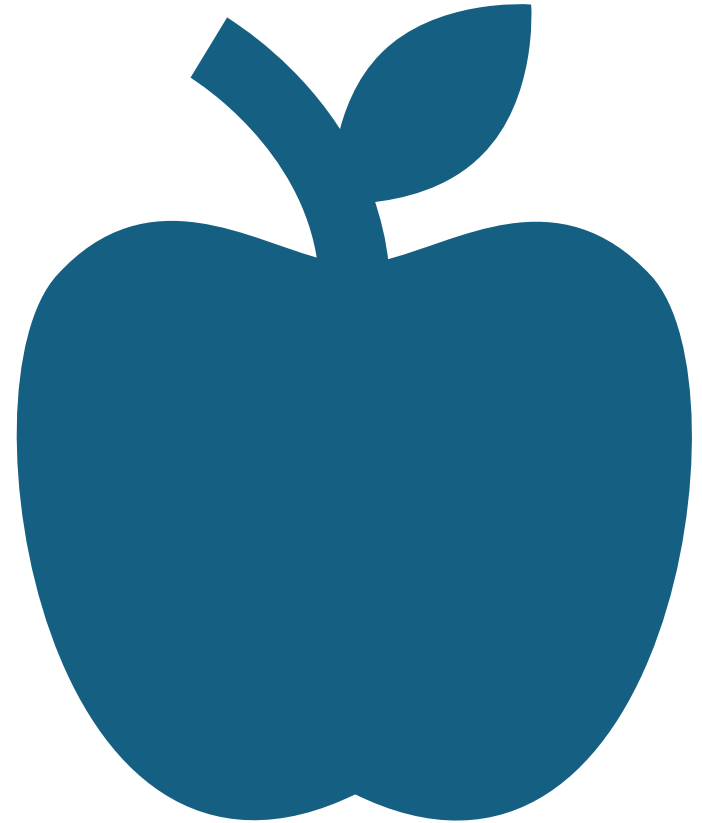
```
print(text.replace("o", ""))        # Removes all 'o' characters
```



# Splitting and Joining string

```
text = "apple,banana,orange"
fruits = text.split(",") # ['apple', 'banana', 'orange']

sentence = ["Python", "is", "fun"]
joined = " ".join(sentence) # "Python is fun"
```



# Trimming Whitespace

```
text = " Hello Python "  
print(text.strip()) # "Hello Python"  
print(text.lstrip()) # "Hello Python "  
print(text.rstrip()) # " Hello Python"
```

# Checking Start/End of String

```
filename = "report.pdf"
```

```
print(filename.startswith("rep")) # True
```

```
print(filename.endswith(".pdf")) # True
```

# Finding and Counting Whitespace

```
text = "Data science with Python and Python is easy"
```

```
print(text.find("Python"))    # 19 (index of first occurrence)
```

```
print(text.rfind("Python"))   # 28 (last occurrence)
```

```
print(text.count("Python"))    # 2
```

# Character Testing Methods

```
a = "123"
```

```
b = "abc"
```

```
c = "Abc123"
```

```
print(a.isdigit()) # True
```

```
print(b.isalpha()) # True
```

```
print(c.isalnum()) # True
```

```
print(" ".isspace()) # True
```

# String Formatting

```
name = "Alice"  
age = 25
```

```
# Using f-strings (Python 3.6+)  
print(f"My name is {name} and I am {age} years old.")
```

```
# Using format()  
print("My name is {} and I am {} years old.".format(name, age))
```

```
# Old style formatting  
print("My name is %s and I am %d years old." % (name, age))
```

# Slicing String

```
text = "PythonProgramming"
```

```
print(text[0:6])  # "Python"
```

```
print(text[6:])   # "Programming"
```

```
print(text[-6:])  # "aming"
```

```
print(text[::2])  # "PtoPormig" (every 2nd character)
```

# Reverse a string

```
text = "Python"  
reversed_text = text[::-1]  
print(reversed_text)    # "nohtyP"
```